

# SIMATS Engineering

## SIMATS Online Programming Lab — Python Programming (Coding Challenge)

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| Course Name     | Python Programming                                                    |
| Course Code     | CSA0804                                                               |
| Name            | Monish.L                                                              |
| Register Number | 1972260035                                                            |
| Topic           | CO2-AT2-CODING CHALLENGE                                              |
| Question        | Q4 — Challenge 4: Advanced Text Statistics Analyser (Hard   20 Marks) |

### Challenge 4: Advanced Text Statistics Analyser [Level: Hard | 20 Marks]

Write a Python program that accepts a paragraph of text and computes: (a) Total words, sentences, and paragraphs. (b) Average word length and average sentence length. (c) The top 5 most frequent words (excluding common stop words: the, is, a, an, in, of, and, to, for). (d) Flesch Reading Ease Score:  $206.835 - 1.015 * (\text{words/sentences}) - 84.6 * (\text{syllables/words})$ . Approximate syllables as the number of vowel groups per word.

#### Constraints

- Sentence ends with . ! or ?
- Handle punctuation attached to words.
- Stop words list is hardcoded in the program.

#### Skills Assessed

- Complex string parsing and tokenization
- Nested loops for syllable counting
- Floating-point arithmetic and rounding

#### Sample Input / Output (as given in the assessment)

Sample Input:  
Python is a powerful programming language...

Expected Output:  
Words: 45 | Sentences: 4 | Paragraphs: 1  
Avg Word Length: 5.2 | Avg Sentence Length: 11.25  
Top 5 Words: python(3), programming(2), language(2), data(2), code(1)  
Flesch Reading Ease: 58.3 (Standard)

**Note on the sample:** the assessment shows the sample input paragraph truncated with "...", so the full original text is not available to reproduce these exact figures. The functions below implement every part of the specification exactly as stated (sentence boundaries on . ! ?, punctuation stripped from words, the

hardcoded stop-word list, vowel-group syllable approximation, and the given Flesch formula) and are verified in the Sample Output section against a separate, fully-specified paragraph instead.

## Approach / Design

- **Paragraphs (split\_paragraphs):** splits the text on blank lines (one or more consecutive newlines) using a regular expression, discarding any empty segments.
- **Sentences (split\_sentences):** splits on '.', '!', or '?' followed by whitespace, so sentence boundaries follow the stated punctuation rule.
- **Words (extract\_words):** uses a regular expression to pull out letter sequences (with apostrophes allowed), automatically stripping attached punctuation like the period in 'language.' and lowercasing each word for consistent counting.
- **Syllable approximation (count\_syllables):** loops through each character of a word, counting a new syllable every time it transitions from a non-vowel to a vowel (a contiguous vowel group), defaulting to 1 for words with no vowels at all.
- **Averages (average\_word\_length / average\_sentence\_length):** compute mean word length in characters and mean sentence length in words, each guarded against division by zero on empty input.
- **Top words (top\_frequent\_words):** loops through the extracted words, skips anything in the hardcoded stop-word set, tallies frequency in a dictionary, and sorts by count (ties broken by first appearance) to return the top 5.
- **Flesch score (flesch\_reading\_ease):** applies the given formula using total words, sentences, and the sum of approximated syllables across all words, rounded to 2 decimal places, with a difficulty label derived from standard Flesch score bands.

## Python Program

```
"""
C02-AT2 - Coding Challenge
Challenge 4: Advanced Text Statistics Analyser [Level: Hard | 20 Marks]
-----
Accepts a paragraph of text and computes:
(a) Total words, sentences, and paragraphs
(b) Average word length and average sentence length
(c) The top 5 most frequent words (excluding common stop words)
(d) Flesch Reading Ease Score, using an approximate syllable
    count based on the number of vowel groups per word

Note on the sample: the assessment's sample input paragraph was
shown truncated ("Python is a powerful programming language...")
so the exact original text is not available to verify figures
against. The functions below implement the specification exactly
as stated (sentence-ending punctuation, punctuation attached to
words, hardcoded stop-word list, and the given Flesch formula) and
are verified below against a separate, fully-specified sample
paragraph instead.
"""

import re

STOP_WORDS = {"the", "is", "a", "an", "in", "of", "and", "to", "for"}
VOWELS = set("aeiou")

# -----
# 1. Paragraph splitting
# -----
def split_paragraphs(text):
```

```

"""
Splits text into paragraphs on blank lines (one or more
consecutive newlines), ignoring empty paragraphs.
"""
raw_parts = re.split(r"\n\s*\n", text.strip())
return [p.strip() for p in raw_parts if p.strip()]

# -----
# 2. Sentence splitting
# -----
def split_sentences(text):
    """
    Splits text into sentences that end with '.', '!', or '?',
    using those punctuation marks (followed by whitespace or end
    of string) as delimiters.
    """
    text = text.strip()
    if not text:
        return []

    raw_sentences = re.split(r"(?<=[.!?])\s+", text)
    return [s.strip() for s in raw_sentences if s.strip()]

# -----
# 3. Word extraction (handles punctuation attached to words)
# -----
def extract_words(text):
    """
    Extracts words from text, stripping attached punctuation
    (e.g. "language." -> "language", "Python's" -> "python's" is
    kept as one token via the apostrophe allowance) and
    lowercasing each word.
    """
    return re.findall(r"[a-zA-Z']+", text.lower())

# -----
# 4. Syllable approximation
# -----
def count_syllables(word):
    """
    Approximates the syllable count of a word as the number of
    contiguous vowel groups (runs of a/e/i/o/u). A word with no
    vowel groups (e.g. all consonants) is counted as 1 syllable.
    """
    groups = 0
    in_vowel_group = False

    for char in word:
        if char in VOWELS:
            if not in_vowel_group:
                groups += 1
                in_vowel_group = True
        else:
            in_vowel_group = False

    return groups if groups > 0 else 1

# -----
# 5. Word length / sentence length averages
# -----
def average_word_length(words):
    if not words:
        return 0.0
    return round(sum(len(w) for w in words) / len(words), 2)
def average_sentence_length(word_count, sentence_count):

```

```

    if sentence_count == 0:
        return 0.0
    return round(word_count / sentence_count, 2)

# -----
# 6. Top 5 most frequent words (excluding stop words)
# -----
def top_frequent_words(words, top_n=5):
    """
    Counts word frequency (excluding stop words) using a loop and
    conditional check, then returns the top_n words sorted by
    frequency (descending), with ties broken by first appearance.
    """
    frequency = {}
    order = []

    for word in words:
        if word in STOP_WORDS:
            continue
        if word not in frequency:
            frequency[word] = 0
            order.append(word)
        frequency[word] += 1

    # sort by frequency descending, preserving first-seen order on ties
    ranked = sorted(order, key=lambda w: (-frequency[w], order.index(w)))
    return [(w, frequency[w]) for w in ranked[:top_n]]

# -----
# 7. Flesch Reading Ease Score
# -----
def flesch_reading_ease(words, sentence_count):
    """
    Computes the Flesch Reading Ease Score using:
    206.835 - 1.015*(words/sentences) - 84.6*(syllables/words)
    Syllables are approximated via count_syllables() per word.
    """
    word_count = len(words)
    if word_count == 0 or sentence_count == 0:
        return 0.0

    total_syllables = sum(count_syllables(w) for w in words)

    score = (
        206.835
        - 1.015 * (word_count / sentence_count)
        - 84.6 * (total_syllables / word_count)
    )
    return round(score, 2)

def reading_ease_label(score):
    """
    Maps a Flesch score to a rough difficulty label.
    """
    if score >= 90:
        return "Very Easy"
    elif score >= 70:
        return "Easy"
    elif score >= 60:
        return "Standard"
    elif score >= 50:
        return "Fairly Difficult"
    elif score >= 30:
        return "Difficult"
    else:
        return "Very Confusing"

```

```

# -----
# 8. Full analysis function
# -----
def analyze_text(text):
    paragraphs = split_paragraphs(text)
    sentences = split_sentences(text)
    words = extract_words(text)

    word_count = len(words)
    sentence_count = len(sentences)
    paragraph_count = len(paragraphs)

    avg_word_len = average_word_length(words)
    avg_sentence_len = average_sentence_length(word_count, sentence_count)
    top_words = top_frequent_words(words)
    flesch_score = flesch_reading_ease(words, sentence_count)

    return {
        "word_count": word_count,
        "sentence_count": sentence_count,
        "paragraph_count": paragraph_count,
        "avg_word_length": avg_word_len,
        "avg_sentence_length": avg_sentence_len,
        "top_words": top_words,
        "flesch_score": flesch_score,
        "flesch_label": reading_ease_label(flesch_score),
    }

# -----
# 9. Report/display function
# -----
def print_report(result):
    print(
        f"Words: {result['word_count']} | "
        f"Sentences: {result['sentence_count']} | "
        f"Paragraphs: {result['paragraph_count']}"
    )
    print(
        f"Avg Word Length: {result['avg_word_length']} | "
        f"Avg Sentence Length: {result['avg_sentence_length']}"
    )

    top_str = ", ".join(f"{w}({c})" for w, c in result["top_words"])
    print(f"Top 5 Words: {top_str}")

    print(
        f"Flesch Reading Ease: {result['flesch_score']} "
        f"({result['flesch_label']})"
    )

# -----
# 10. Main driver
# -----
def main():
    sample_text = (
        "Python is a powerful programming language. It is widely used for "
        "web development, data science, and automation. Many developers "
        "love Python because the syntax is simple and readable. Python "
        "code is often shorter than code written in other languages."
    )

    result = analyze_text(sample_text)
    print_report(result)

if __name__ == "__main__":
    main()

```

## Sample Output (verification run on a fully-specified paragraph)

**Input used:** "Python is a powerful programming language. It is widely used for web development, data science, and automation. Many developers love Python because the syntax is simple and readable. Python code is often shorter than code written in other languages."

```
Words: 39 | Sentences: 4 | Paragraphs: 1
Avg Word Length: 5.26 | Avg Sentence Length: 9.75
Top 5 Words: python(3), code(2), powerful(1), programming(1), language(1)
Flesch Reading Ease: 40.75 (Difficult)
```

## Conclusion

The analyser tokenizes a paragraph into paragraphs, sentences, and words using targeted regular expressions that respect the stated sentence-ending punctuation and strip punctuation attached to words. Syllables are approximated by counting contiguous vowel groups per word, which feeds directly into the given Flesch Reading Ease formula alongside the word and sentence counts. Word frequency excludes the hardcoded stop-word list before ranking the top 5. Running the program on the verification paragraph above produces internally consistent, correctly rounded statistics, confirming the implementation is sound even though the assessment's own sample paragraph was not given in full.